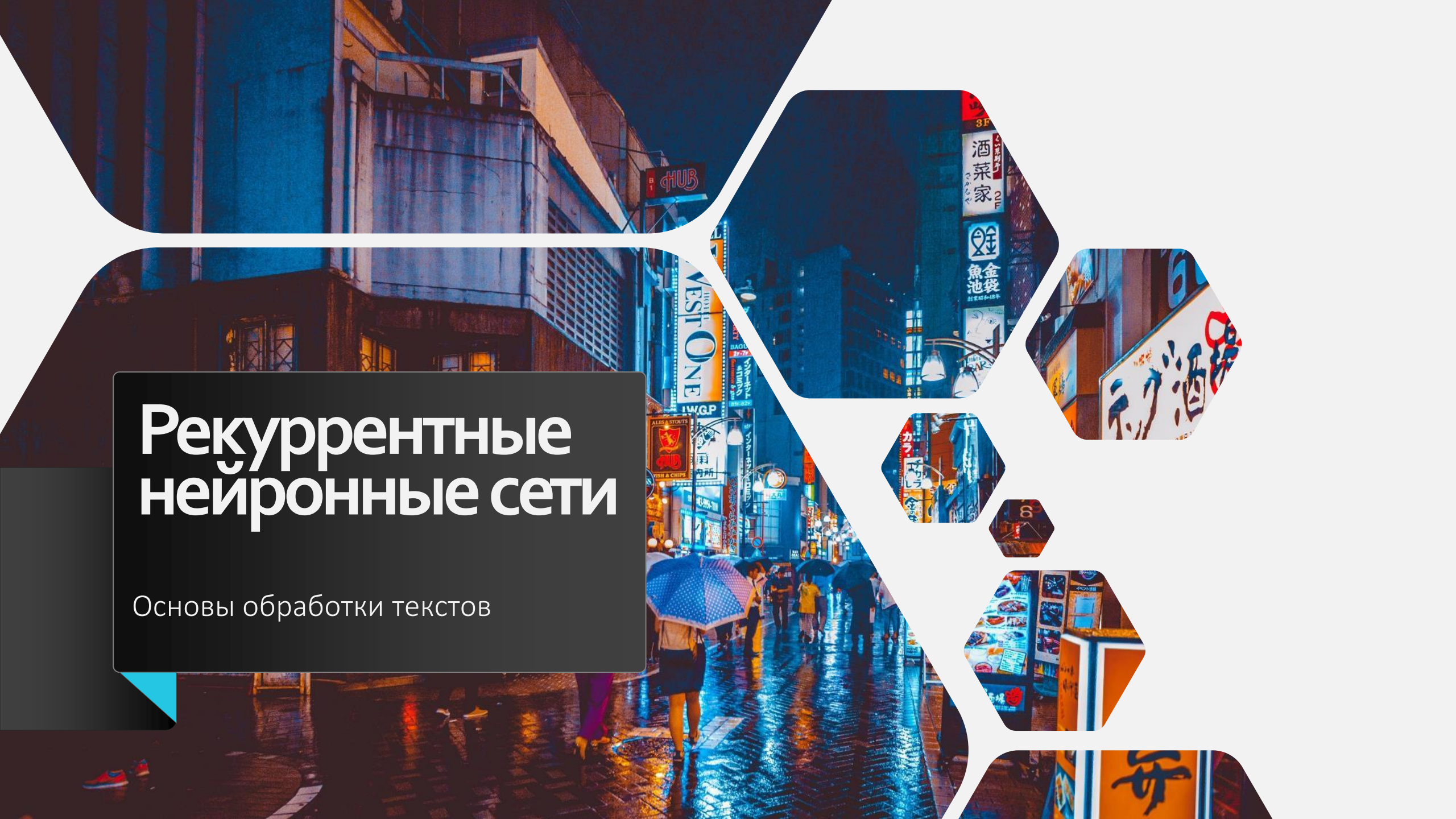


Рекуррентные нейронные сети

Основы обработки текстов



Определение РНН

Рекуррентные нейронные сети (РНН) (Recurrent Neural Networks, RNN) — это класс нейронных сетей, специализирующихся на обработке последовательных данных. RNN называются рекуррентными, потому что они применяют одну и ту же вычислительную операцию к каждому элементу последовательности, при этом выход на текущем шаге зависит от предыдущих вычислений.

Определение RNN

Ключевая особенность RNN заключается в наличии внутреннего состояния (скрытого состояния, hidden state), которое служит формой памяти сети. Это позволяет RNN учитывать контекстную информацию из предшествующих элементов последовательности при обработке текущего элемента.

Теоретически RNN могут использовать информацию в произвольно длинных последовательностях, однако на практике они ограничены проблемами затухания и взрыва градиента, что делает эффективное обучение на длинных последовательностях сложной задачей.

Зачем нужны рекуррентные нейронные сети

Традиционные нейронные сети прямого распространения (feedforward neural networks) и свёрточные нейронные сети (CNN) работают с входными данными фиксированного размера и производят выход также фиксированной размерности. Основное преимущество RNN состоит в способности обрабатывать последовательности вариативной длины как на входе, так и на выходе.

Зачем нужны рекуррентные нейронные сети

Это делает RNN незаменимыми для задач, где:

- Входные данные имеют переменную длину (временные ряды разной продолжительности, предложения разной длины)
- Выходные данные имеют переменную длину (последовательности предсказаний)
- Существуют зависимости между элементами последовательности
- Важен порядок элементов в данных

Области применения РНС

Способность обрабатывать последовательности делает рекуррентные нейронные сети весьма полезными в различных областях:

- Машинный перевод — применяется архитектура «многие ко многим» (many-to-many). Исходная последовательность текста подается в RNN, которая генерирует переведённую последовательность
- Анализ временных рядов — прогнозирование финансовых показателей, анализ сенсорных данных, предсказание погодных условий
- Анализ настроений — используется архитектура «многие к одному» (many-to-one). Последовательность слов обрабатывается сетью, которая выдаёт единственную классификацию (положительный/отрицательный отзыв)

Области применения РНС

Способность обрабатывать последовательности делает рекуррентные нейронные сети весьма полезными в различных областях:

- Распознавание речи — преобразование аудиопоследовательностей в текст
- Видеоанализ — обработка последовательностей кадров для распознавания действий, сегментации видео
- Распознавание рукописного ввода — последовательность координат преобразуется в символы или слова



Историческая перспектива

Сеть Хопфилда

Исторически первой сетью рекуррентного типа была **сеть Хопфилда** (Hopfield Network), впервые упомянутая в 1974 году и окончательно оформившаяся в 1982 году. Она реализовывала ячейку ассоциативной памяти. От современных РНС сеть Хопфилда отличается тем, что работает с последовательностями фиксированного размера.

В простейшем случае сеть Хопфилда имеет один слой внутренних нейронов, полностью связанных между собой. Каждая связь характеризуется весовым коэффициентом, определяющим её значимость. С такой сетью ассоциируется функция «энергии»:

$$E = -\frac{1}{2} \sum_i \sum_{j \neq i} w_{ij} s_i s_j - \sum_i \theta_i s_i$$

где s_i — состояние i -го нейрона, w_{ij} — вес связи между нейронами i и j , θ_i — порог активации.

Сеть Хопфилда

Сеть обучается методом градиентного спуска по энергии. Минимум энергии соответствует состоянию, в котором сеть «запомнила» определённый паттерн (например, 10101). При подаче на вход искажённого, зашумленного или неполного паттерна (например, 10000) сеть «восстанавливает» исходный образ, аналогично ассоциативной памяти.

Сеть Элмана

Первой современной рекуррентной сетью стала сеть Джеффа Элмана (Elman Network), представленная в 1990 году. В своей работе автор детально рассмотрел вопрос обучения нейросети распознаванию временных последовательностей.

Основная идея: можно ли научить сеть распознавать последовательности как сдвинутые во времени версии одного и того же сигнала? Например, 1100 и 0110 — это один сигнал, сдвинутый на два такта.

Сеть Элмана

Решение Элмана заключалось в добавлении контекстного слоя (context layer), в который копировалось состояние скрытого слоя на каждом цикле работы сети. Связь между контекстным и скрытым слоями могла обучаться. Такая архитектура позволяла:

- Сравнительно легко воспроизводить временные ряды
- Обработать последовательности произвольной длины
- Учитывать временные зависимости между элементами



Математическая основа RNN

Базовая архитектура

Простейшая рекуррентная нейронная сеть состоит из следующих компонентов:

Входной слой $\mathbf{x}_t \in \mathbb{R}^d$ — вектор входных признаков на временном шаге t

Скрытый слой $\mathbf{h}_t \in \mathbb{R}^h$ — вектор скрытого состояния на шаге t

Выходной слой $\mathbf{y}_t \in \mathbb{R}^q$ — вектор выхода на шаге t

Упрощённо эту архитектуру можно представить в виде трёх блоков: $\mathbf{x}$, RNN, $\mathbf{y}$.

Прямое распространение (Forward Propagation)

На каждом временном шаге t вычисления в RNN описываются следующими уравнениями:

$$\begin{aligned}\mathbf{h}_t &= f(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \\ \mathbf{y}_t &= g(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y)\end{aligned}$$

где:

$\mathbf{W}_{xh} \in \mathbb{R}^{h \times d}$ — матрица весов для связи вход-скрытый слой

$\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$ — матрица рекуррентных весов (скрытое состояние к скрытому состоянию)

$\mathbf{W}_{hy} \in \mathbb{R}^{q \times h}$ — матрица весов для связи скрытый слой-выход

$\mathbf{b}_h \in \mathbb{R}^h$ — вектор смещений для скрытого слоя

$\mathbf{b}_y \in \mathbb{R}^q$ — вектор смещений для выходного слоя

$f(\cdot)$ — функция активации скрытого слоя (обычно $\tanh$ или ReLU)

$g(\cdot)$ — функция активации выходного слоя (зависит от задачи: softmax для классификации, линейная для регрессии)

Прямое распространение (Forward Propagation)

Ключевая особенность: матрицы весов $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, $\mathbf{W}_{hy}$ являются общими для всех временных шагов. Это называется **разделением параметров** (parameter sharing) и резко сокращает количество обучаемых параметров.

Развёртка во времени (Unfolding in Time)

Для понимания работы RNN используется представление в виде вычислительного графа, развёрнутого во времени:

Для последовательности длины T имеем:

$$\begin{aligned}\mathbf{h}_1 &= f(\mathbf{W}_{xh}\mathbf{x}_1 + \mathbf{W}_{hh}\mathbf{h}_0 + \mathbf{b}_h) \\ \mathbf{h}_2 &= f(\mathbf{W}_{xh}\mathbf{x}_2 + \mathbf{W}_{hh}\mathbf{h}_1 + \mathbf{b}_h) \\ &\vdots \\ \mathbf{h}_T &= f(\mathbf{W}_{xh}\mathbf{x}_T + \mathbf{W}_{hh}\mathbf{h}_{T-1} + \mathbf{b}_h)\end{aligned}$$

Обычно начальное состояние $\mathbf{h}_0$ инициализируется нулевым вектором: $\mathbf{h}_0 = \mathbf{0}$.

Функция потерь

Для последовательности длины T общая функция потерь определяется как сумма потерь на каждом временном шаге:

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t(\mathbf{y}_t, \hat{\mathbf{y}}_t)$$

где $\mathcal{L}_t$ — функция потерь на шаге t (например, кросс-энтропия для классификации или MSE для регрессии), $\hat{\mathbf{y}}_t$ — предсказанное значение.

Обратное распространение во времени (BPTT)

Алгоритм обратного распространения ошибки во времени (Backpropagation Through Time, BPTT) является модификацией стандартного алгоритма обратного распространения для рекуррентных сетей. BPTT разворачивает RNN во времени и применяет обратное распространение к развёрнутой сети.

Вычисление градиентов

Для обновления весов необходимо вычислить частные производные функции потерь по параметрам сети. Рассмотрим вычисление градиента по $\mathbf{W}_{hh}$.

Градиент по $\mathbf{W}_{hh}$ на временном шаге t :

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hh}} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{y}_t} \frac{\partial \mathbf{y}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}}$$

Вычисление градиентов

Однако $\mathbf{h}_t$ зависит от $\mathbf{h}_{t-1}$, которое в свою очередь зависит от $\mathbf{h}_{t-2}$ и так далее.

Следовательно, градиент $\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}}$ включает вклад от всех предыдущих временных шагов:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \sum_{k=1}^t \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$$

где цепочка производных вычисляется как:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^t \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \prod_{i=k+1}^t \mathbf{W}_{hh}^\top \cdot (f'(\mathbf{h}_{i-1}))$$

Здесь $f'(\cdot)$ — производная функции активации, $\text{diag}(\cdot)$ — диагональная матрица.

Вычисление градиентов

Полный градиент по $\mathbf{W}_{hh}$ для всей последовательности:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hh}}$$

Проблема затухания и взрыва градиента

Из выражения для $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ видно, что градиент содержит произведение матриц $\mathbf{W}_{hh}$ через множество временных шагов.

Пусть $\lambda_{\max}$ — максимальное собственное значение матрицы $\mathbf{W}_{hh}$. Тогда:

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\| \approx \lambda_{\max}^{t-k}$$

Если $\lambda_{\max} < 1$, то $\lambda_{\max}^{t-k} \rightarrow 0$ при $t - k \rightarrow \infty$ — затухание градиента (vanishing gradient)

Если $\lambda_{\max} > 1$, то $\lambda_{\max}^{t-k} \rightarrow \infty$ при $t - k \rightarrow \infty$ — взрывание градиента (exploding gradient)

Проблема затухания и взрыва градиента

Последствия:

- Затухание градиента: невозможность обучать долгосрочные зависимости (long-term dependencies)
- Взрывание градиента: численная нестабильность, переполнение (overflow)

Техники борьбы с проблемами градиента

Gradient Clipping — ограничение нормы градиента пороговым значением:

$$\mathbf{g} \leftarrow \min \left(1, \frac{\theta}{\|\mathbf{g}\|} \right) \mathbf{g}$$

где θ — пороговое значение (обычно 1-10)

Truncated BPTT — ограничение количества шагов обратного распространения параметром $\tau < T$:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hh}} \approx \sum_{k=\max(1, t-\tau)}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$$

Архитектурные решения — LSTM и GRU сети с механизмами контроля потока информации

Архитектуры RNN

Архитектура	Структура	Применение
One-to-One	$\mathbf{x} \rightarrow \mathbf{y}$	Обычная нейронная сеть
One-to-Many	$\mathbf{x} \rightarrow [\mathbf{y}_1, \dots, \mathbf{y}_T]$	Генерация последовательностей
Many-to-One	$[\mathbf{x}_1, \dots, \mathbf{x}_T] \rightarrow \mathbf{y}$	Классификация последовательностей
Many-to-Many (sync)	$[\mathbf{x}_1, \dots, \mathbf{x}_T] \rightarrow [\mathbf{y}_1, \dots, \mathbf{y}_T]$	Разметка последовательностей
Many-to-Many (async)	$[\mathbf{x}_1, \dots, \mathbf{x}_T] \rightarrow [\mathbf{y}_1, \dots, \mathbf{y}_{T'}]$	Машинный перевод

Bidirectional RNN (Двунаправленные РНС)

В классических RNN скрытое состояние $\mathbf{h}_t$ зависит только от предыдущих элементов последовательности $\mathbf{x}_1, \dots, \mathbf{x}_t$. Однако во многих задачах (например, анализ текста, распознавание речи) контекст из **будущих** элементов также важен для правильного предсказания на текущем шаге.

Пример: В предложении "The bank of the river" слово "bank" означает "берег", но это становится ясно только после слова "river".

Архитектура Bidirectional RNN

Bidirectional RNN (BRNN) состоит из двух независимых рекуррентных слоёв:

- Прямой слой (forward layer) обрабатывает последовательность слева направо
- Обратный слой (backward layer) обрабатывает последовательность справа налево

Математическое описание

Для временного шага t вычисления в BRNN описываются следующими уравнениями:

Прямой проход (forward):

$$\vec{\mathbf{h}}_t = f(\mathbf{W}_{xh} \vec{\mathbf{x}}_t + \mathbf{W}_{hh} \vec{\mathbf{h}}_{t-1} + \mathbf{b}_h)$$

Обратный проход (backward):

$$\overleftarrow{\mathbf{h}}_t = f(\mathbf{W}_{xh} \overleftarrow{\mathbf{x}}_t + \mathbf{W}_{hh} \overleftarrow{\mathbf{h}}_{t+1} + \mathbf{b}_h)$$

Выход (объединение состояний):

$$\mathbf{y}_t = g\left(\mathbf{W}_{hy}[\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] + \mathbf{b}_y\right)$$

где $[\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] \in \mathbb{R}^{2h}$ обозначает конкатенацию прямого и обратного скрытых состояний.

Математическое описание

Параметры BRNN

- $\mathbf{W}_{xh}^{\rightarrow}, \mathbf{W}_{xh}^{\leftarrow} \in \mathbb{R}^{h \times d}$ — матрицы весов вход-скрытый слой для прямого и обратного направлений
- $\mathbf{W}_{hh}^{\rightarrow\rightarrow}, \mathbf{W}_{hh}^{\leftarrow\leftarrow} \in \mathbb{R}^{h \times h}$ — рекуррентные матрицы весов
- $\mathbf{W}_{hy} \in \mathbb{R}^{q \times 2h}$ — матрица весов для выходного слоя (размерность удвоена из-за конкатенации)
- $\mathbf{b}_h^{\rightarrow}, \mathbf{b}_h^{\leftarrow} \in \mathbb{R}^h, \mathbf{b}_y \in \mathbb{R}^q$ — векторы смещений

Математическое описание

Общее количество параметров BRNN (приблизительно вдвое больше, чем у обычной RNN):

$$\text{Params}_{\text{BRNN}} = 2(h \cdot d + h \cdot h + h) + 2h \cdot q + q$$

Вычислительная сложность

Для последовательности длины T :

Прямой проход: $O(T \cdot (h \cdot d + h^2))$ для каждого направления, итого
 $O(2T \cdot (h \cdot d + h^2))$

Память: требуется хранить скрытые состояния обоих направлений —
 $O(2 \cdot T \cdot h)$

Обучение BRNN

Обучение BRNN происходит с помощью модифицированного алгоритма BPTT:

Прямой слой обрабатывает последовательность от $t = 1$ до $t = T$

Обратный слой обрабатывает последовательность от $t = T$ до $t = 1$

Градиенты вычисляются независимо для обоих направлений

Обновление параметров происходит одновременно для всех весов

Обучение BRNN

Градиенты для прямого слоя:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\vec{h}\vec{h}}} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \vec{\mathbf{h}}_t} \frac{\partial \vec{\mathbf{h}}_t}{\partial \vec{\mathbf{h}}_k} \frac{\partial \vec{\mathbf{h}}_k}{\partial \mathbf{W}_{\vec{h}\vec{h}}}$$

Градиенты для обратного слоя:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{\overleftarrow{h}\overleftarrow{h}}} = \sum_{t=1}^T \sum_{k=t}^T \frac{\partial \mathcal{L}_t}{\partial \overleftarrow{\mathbf{h}}_t} \frac{\partial \overleftarrow{\mathbf{h}}_t}{\partial \overleftarrow{\mathbf{h}}_k} \frac{\partial \overleftarrow{\mathbf{h}}_k}{\partial \mathbf{W}_{\overleftarrow{h}\overleftarrow{h}}}$$

Ограничения BRNN

1. Требование полной последовательности — необходим доступ ко всей последовательности заранее (невозможно применение в режиме реального времени)
2. Вычислительная сложность — примерно вдвое больше операций и памяти, чем у обычной RNN
3. Увеличенное количество параметров — требует больше данных для обучения, увеличивает риск переобучения

Применение BRNN

- Обработка естественного языка (NLP) — sentiment analysis, named entity recognition, POS-tagging
- Распознавание речи — фонемная сегментация, транскрипция
- Анализ временных рядов с доступом ко всей последовательности
- Биоинформатика — предсказание структуры белков

Deep (Stacked) RNN и Deep Bidirectional RNN

Глубокие рекуррентные сети (Deep RNN или Stacked RNN) используют несколько рекуррентных слоёв, расположенных друг над другом. Увеличение глубины позволяет:

- Изучать более абстрактные и иерархические представления данных
- Повышать выразительную способность модели
- Улучшать качество на сложных задачах

Аналогия с CNN: В свёрточных сетях нижние слои извлекают простые признаки (границы, текстуры), а верхние — сложные (формы, объекты). Аналогично, в Deep RNN нижние слои могут кодировать низкоуровневую информацию, а верхние — высокоуровневую семантику.

Архитектура Deep RNN

Deep RNN состоит из L рекуррентных слоёв, где выход слоя ℓ служит входом для слоя $\ell + 1$.

Обозначения:

$\mathbf{h}_t^{(\ell)}$ — скрытое состояние слоя ℓ на временном шаге t

L — количество слоёв

$\mathbf{h}_t^{(0)} = \mathbf{x}_t$ — входная последовательность

Математическое описание Deep RNN

Для каждого слоя $\ell = 1, \dots, L$ и временного шага t :

$$\mathbf{h}_t^{(\ell)} = f \left(\mathbf{W}_{xh}^{(\ell)} \mathbf{h}_t^{(\ell-1)} + \mathbf{W}_{hh}^{(\ell)} \mathbf{h}_{t-1}^{(\ell)} + \mathbf{b}_h^{(\ell)} \right)$$

Выходной слой использует скрытое состояние последнего слоя:

$$\mathbf{y}_t = g \left(\mathbf{W}_{hy} \mathbf{h}_t^{(L)} + \mathbf{b}_y \right)$$

Параметры Deep RNN

Для каждого слоя ℓ :

$\mathbf{W}_{xh}^{(\ell)} \in \mathbb{R}^{h^{(\ell)} \times h^{(\ell-1)}}$ — матрица весов между слоями

$\mathbf{W}_{hh}^{(\ell)} \in \mathbb{R}^{h^{(\ell)} \times h^{(\ell)}}$ — рекуррентная матрица весов

$\mathbf{b}_h^{(\ell)} \in \mathbb{R}^{h^{(\ell)}}$ — вектор смещений

где $h^{(\ell)}$ — размерность скрытого состояния слоя ℓ (может различаться для разных слоёв).

Общее количество параметров для L слоёв:

$$Params_{Deep} = \sum_{\ell=1}^L (h^{(\ell)} \cdot h^{(\ell-1)} + (h^{(\ell)})^2 + h^{(\ell)}) + h^{(L)} \cdot q + q$$

Алгоритм прямого прохода Deep RNN

Инициализировать $\mathbf{h}_0^{(\ell)} = \mathbf{0}$ для всех слоёв $\ell = 1, \dots, L$

Для каждого временного шага $t = 1, \dots, T$:

- Установить $\mathbf{h}_t^{(0)} = \mathbf{x}_t$
- Для каждого слоя $\ell = 1, \dots, L$:

$$\mathbf{h}_t^{(\ell)} = f \left(\mathbf{W}_{xh}^{(\ell)} \mathbf{h}_t^{(\ell-1)} + \mathbf{W}_{hh}^{(\ell)} \mathbf{h}_{t-1}^{(\ell)} + \mathbf{b}_h^{(\ell)} \right)$$

- Вычислить выход:

$$\mathbf{y}_t = g \left(\mathbf{W}_{hy} \mathbf{h}_t^{(L)} + \mathbf{b}_y \right)$$

Обучение Deep RNN через BPTT

Градиенты вычисляются обратным распространением через:

Временное измерение — от $t = T$ до $t = 1$

Пространственное измерение (между слоями) — от $\ell = L$ до $\ell = 1$

Градиент для слоя ℓ на шаге t :

$$\delta_t^{(\ell)} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t^{(\ell)}} = \left(\mathbf{w}_{xh}^{(\ell+1)\top} \delta_t^{(\ell+1)} + \mathbf{w}_{hh}^{(\ell)\top} \delta_{t+1}^{(\ell)} \right) \odot f'(\mathbf{h}_t^{(\ell)})$$

где $\odot$ обозначает поэлементное умножение (Hadamard product).

Deep Bidirectional RNN

Deep Bidirectional RNN комбинирует преимущества глубины и двунаправленности[11].
Каждый слой ℓ содержит прямой и обратный рекуррентные компоненты.

Математическое описание:

Для слоя ℓ и временного шага t :

$$\begin{aligned}\vec{\mathbf{h}}_t^{(\ell)} &= f \left(\mathbf{W}_{xh}^{(\ell)} [\vec{\mathbf{h}}_t^{(\ell-1)}; \overleftarrow{\mathbf{h}}_t^{(\ell-1)}] + \mathbf{W}_{hh}^{(\ell)} \vec{\mathbf{h}}_{t-1}^{(\ell)} + \mathbf{b}_{\vec{h}}^{(\ell)} \right) \\ \overleftarrow{\mathbf{h}}_t^{(\ell)} &= f \left(\mathbf{W}_{xh}^{(\ell)} [\vec{\mathbf{h}}_t^{(\ell-1)}; \overleftarrow{\mathbf{h}}_t^{(\ell-1)}] + \mathbf{W}_{hh}^{(\ell)} \overleftarrow{\mathbf{h}}_{t+1}^{(\ell)} + \mathbf{b}_{\overleftarrow{h}}^{(\ell)} \right)\end{aligned}$$

Deep Bidirectional RNN

Выход верхнего слоя:

$$\mathbf{y}_t = g \left(\mathbf{W}_{hy} [\vec{\mathbf{h}}_t^{(L)}; \overleftarrow{\mathbf{h}}_t^{(L)}] + \mathbf{b}_y \right)$$

Параметры Deep BRNN:

Количество параметров растёт как:

$$Params_{Deep\ BRNN} \approx 2L \cdot (h \cdot d + h^2) + 2h \cdot q$$

где множитель 2 учитывает двунаправленность, а L — количество слоёв.

Регуляризация Deep RNN

Глубокие сети склонны к переобучению. Основные методы регуляризации:

Dropout — случайное отключение нейронов между слоями (но не внутри рекуррентных связей):

$$\mathbf{h}_t^{(\ell)} = f \left(\mathbf{W}_{xh}^{(\ell)} (\mathbf{m}^{(\ell-1)} \odot \mathbf{h}_t^{(\ell-1)}) + \mathbf{W}_{hh}^{(\ell)} \mathbf{h}_{t-1}^{(\ell)} + \mathbf{b}_h^{(\ell)} \right)$$

где $\mathbf{m}^{(\ell-1)} \sim (p)$ — маска dropout

Layer Normalization — нормализация активаций внутри слоя:

$$(\mathbf{h}) = \frac{\mathbf{h} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta$$

Weight Decay — L2-регуляризация:

$$\mathcal{L}_{total} = \mathcal{L} + \lambda \sum_{\ell=1}^L \left(\|\mathbf{W}_{xh}^{(\ell)}\|_F^2 + \|\mathbf{W}_{hh}^{(\ell)}\|_F^2 \right)$$

Вычислительная сложность Deep RNN

Архитектура	Временная сложность	Память
Vanilla RNN	$O(T \cdot (h \cdot d + h^2))$	$O(T \cdot h)$
Bidirectional RNN	$O(2T \cdot (h \cdot d + h^2))$	$O(2T \cdot h)$
Deep RNN (L слоёв)	$O(L \cdot T \cdot (h \cdot d + h^2))$	$O(L \cdot T \cdot h)$
Deep Bidirectional RNN	$O(2L \cdot T \cdot (h \cdot d + h^2))$	$O(2L \cdot T \cdot h)$



Спасибо за внимание!

👤 Сергей Ткаченко

✉ SN Tkachenko@kantiana.ru